

OWASP Top 10

Software Supply Chain Failures

BIG school

BIGSEIO

GENERAMOS
NEGOCIO

Software Supply Chain Failures

Índice

- Introducción
- ¿Qué significa “Software Supply Chain Failures”?
- ¿Por qué ocurre esta vulnerabilidad?
- Ejemplos reales y típicos
- ¿Qué puede hacer un atacante si explota esto?
- ¿Cómo detectarlo?
- ¿Cómo prevenirlo?
- ¿Cómo mitigar si ya estás afectado?
- Checklist rápido
- Conclusiones



Software Supply Chain Failures

BIG school

Introducción

- La cadena de suministro incluye:
librerías, frameworks, contenedores,
herramientas CI/CD y registros públicos
- Riesgo silencioso: vulnerabilidades
heredadas o código malicioso en
componentes de terceros
- Objetivo: entender, detectar y prevenir
fallos que nacen fuera de nuestro código
pero impactan nuestra aplicación

¿Qué significa "Software Supply Chain Failures"?

- Ocurre cuando dependemos de componentes vulnerables, comprometidos o sin verificación de integridad
- No es un bug en tu código: es confiar ciegamente en una dependencia, imagen o pipeline alterado
- Ejemplo: instalar un paquete de npm/PyPI que parece legítimo pero ejecuta código malicioso en build o run-time

¿Por qué ocurre esta vulnerabilidad?

- **Confianza por defecto en registros públicos sin verificar provenance o firma digital**
- **Dependencias transitivas ocultas: una librería trae otra, y nadie audita la cadena completa**
- **Pipelines CI/CD sobreprivilegiados, con secretos expuestos o scripts de terceros no validados**
- **Actualizaciones automáticas sin revisión humana: si el registro es comprometido, el malware entra directo a producción**

Ejemplos reales y típicos

- Typosquatting: paquetes con nombres similares a legítimos (react-d0m, lodahs)
- Cuentas de mantenedores comprometidas: como event-stream o ua-parser-js
- Dependencias transitivas envenenadas: vulnerabilidad oculta en una dependencia de tercer nivel
- Pipelines inseguros: workflows de CI/CD con permisos excesivos y sin validación de código
- Imágenes sin firmar: docker pull de fuentes no oficiales sin verificar digest

¿Qué puede hacer un atacante si explota esto?

- Inyectar código malicioso que se ejecuta en build-time o run-time
- Robar secretos, tokens o credenciales almacenadas en entornos de CI/CD
- Distribuir malware a miles de proyectos que dependen de un paquete comprometido
- Escalar privilegios y moverse lateralmente desde tu infraestructura hacia otros sistemas
- Comprometer la integridad de tu software si roban claves de firma de código

¿Cómo detectarlo?

- **SBOM (Software Bill of Materials):** inventario exacto de componentes con Syft, CycloneDX o SPDX
- **Escaneo automatizado:** Snyk, Dependabot, Trivy o Gype para CVEs y paquetes sospechosos
- **Verificación de integridad:** firmas digitales, cosign, in-toto o SLSA para validar artefactos
- **Auditoría de CI/CD:** revisar permisos, exposición de secretos y ejecución de scripts de terceros
- **Monitorización continua:** alertas ante nuevas versiones, paquetes yanked o reportes de malware

¿Cómo prevenirlo?

- Fijar versiones y usar lockfiles (package-lock.json, poetry.lock, go.sum)
- Generar y almacenar un SBOM actualizado por cada release
- Firmar contenedores, binarios y paquetes con cosign o GPG
- Implementar SLSA o marcos de provenance para validar origen del código
- Configurar Dependabot/Renovate con revisión humana antes de merge
- Usar registros privados o proxies (Artifactory/Nexus) con escaneo previo para producción
- Rotar y proteger secretos de CI/CD con vaults centralizados

¿Cómo mitigar si ya estás afectado?

- Aislar inmediatamente: detener pipelines afectados y revertir a versiones verificadas
- Identificar alcance: usar SBOM para rastrear artefactos, entornos o clientes impactados
- Rotar secretos expuestos: tokens, claves API, credenciales de CI/CD y firmas de código
- Aplicar controles compensatorios: WAF, políticas de red estrictas, deshabilitar funcionalidades no críticas
- Auditar logs y tráfico: buscar exfiltración, conexiones sospechosas o scripts no autorizados
- Notificar y colaborar: informar a usuarios/downstream y reportar el paquete al registro

Checklist rápido

1. ¿Tengo un SBOM actualizado de mi aplicación y sus dependencias?
2. ¿Uso lockfiles y versiones fijas en lugar de rangos abiertos (* o latest)?
3. ¿Escaneo automáticamente dependencias y busco paquetes maliciosos, no solo CVEs?
4. ¿Mis artefactos y contenedores están firmados digitalmente?
5. ¿Mis pipelines CI/CD tienen permisos mínimos y secretos protegidos?
6. ¿Uso registros curados o proxies con escaneo previo para producción?
7. ¿Verifico reputación y mantenimiento de cada librería antes de instalarla?
8. ¿Tengo alertas para actualizaciones sospechosas o paquetes yanked?

Conclusiones

- La cadena de suministro es invisible pero crítica: verificar, no solo confiar
- Automatizar detección, trazabilidad y firma reduce riesgo sin frenar el desarrollo
- Un SBOM + escaneo continuo + políticas de integridad = defensa proactiva
- Como desarrollador: fijar versiones, revisar dependencias y exigir provenance te hace indispensable
- Mantener tu supply chain segura es higiene digital: simple, repetible y de alto impacto



¡MUCHAS GRACIAS!

BIG school